NLP with Deep Learning

Mini-Project 1

A Retrieval-Augmented Generation System for
Sindh Criminal Prosecution Law

Group Members

Tayyib Bin Hisham Sayyid	**29087**
Syed Rameez Ul Hoda	**29057**
Zuhair Amirali Merchant	**29021**

Department of Computer Science
Institute of Business Administration

April 5, 2026

Abstract

Criminal prosecution in Sindh is governed by a dense and fragmented body of primary legislation — the Sindh Criminal Court Rules 2012, the Sindh Criminal Prosecution Service Act 2009, and a sequence of amendment Acts spanning 2011, 2014, 2015, and 2025 — much of which is scattered across official gazettes, High Court publications, and unindexed PDFs. Practising prosecutors, defence advocates, and law clerks routinely need to answer precise procedural questions ("what are the qualifications for appointment of the Prosecutor General?", "what is the procedure for recording a confession?", "what rules govern the production of digital video evidence?") but must currently locate the answer by manually searching multiple documents, cross-referencing amendments, and reconciling consolidated versions. This is slow, error-prone, and poorly suited to the tight timelines of active litigation. We chose Sindh Criminal Prosecution Law as our project domain precisely because it exhibits this combination — high procedural specificity, heavy reliance on exact section numbers, and no publicly available purpose-built search tool — which makes it an ideal target for a retrieval-augmented generation system. A well-grounded RAG system offers lawyers a concrete productivity gain: it returns the governing provision verbatim, cites the exact rule or section, and refuses to fabricate content when the source is silent, yielding answers a practitioner can confidently carry into court.

This report documents the design, implementation, and empirical evaluation of a Retrieval-Augmented Generation (RAG) question-answering system for the Sindh Criminal Prosecution Law corpus. The system integrates a cloud-hosted vector database (Pinecone), lightweight on-device sentence embeddings (`all-MiniLM-L6-v2`), hybrid retrieval combining BM25 keyword search with semantic search under Reciprocal Rank Fusion, cross-encoder reranking, and a large language model (`llama-3.3-70b-versatile`) served via the Groq Inference API. The interface is a Streamlit application deployed publicly on Hugging Face Spaces. An automated LLM-as-a-Judge evaluation pipeline measures Faithfulness (via claim extraction and verification) and Relevancy (via alternate query generation and cosine similarity) on a fixed test set of fifteen legal queries. An ablation study comparing two chunking strategies against two retrieval modes (sixty query–configuration pairs in total) empirically identifies the best pipeline. The final system achieves a Faithfulness score of 92.4% and a Relevancy score of 82.6% on answerable queries, validating the hybrid retrieval design for the legal-text domain.

Contents

1 Platform Details **3**
- 1.1 Execution Environments . . . 3
- 1.2 Hosting and Public Access . . . 3
- 1.3 Division of Computation . . . 3

2 Data Details **4**
- 2.1 Source and Authority . . . 4
- 2.2 Corpus Inventory . . . 4
- 2.3 Dataset Size and Chunk Statistics . . . 4
- 2.4 Known Data-Quality Limitations . . . 5

3 Algorithms, Models, and Retrieval Methods **6**
- 3.1 Chunking Strategy . . . 6
- 3.2 Embedding Model . . . 6
- 3.3 Retrieval Stack: Hybrid Search with RRF and Cross-Encoder Reranking . . . 6
- 3.4 Confidence Gate . . . 7
- 3.5 LLM and Inference Backend . . . 7
- 3.6 Prompt Structures . . . 8

4 Performance Metrics and Ablation Study **10**
- 4.1 Evaluation Methodology . . . 10
- 4.2 Primary Evaluation (`evaluate_rag.py`) . . . 10
- 4.3 Ablation Study (`ablation_study.py`) . . . 10
- 4.4 Claim Extraction and Verification Examples . . . 11
- 4.5 Alternate Query Generation Examples (Relevancy) . . . 12
- 4.6 Computational Efficiency . . . 12
- 4.7 Insights and Lessons Learned . . . 12

5 Best Model Selection **15**

6 Reproducibility **16**
- 6.1 Environment and Dependencies . . . 16
- 6.2 API Keys . . . 16
- 6.3 End-to-End Reproduction Steps . . . 16
- 6.4 Artefacts Produced by the Pipeline . . . 17

7 System Demonstration **18**

1 Platform Details

The RAG pipeline for this project was intentionally partitioned across multiple environments to separate heavy one-off preprocessing from lightweight, stateless query serving. This architectural choice reflects a production-oriented deployment pattern in which expensive artifacts (chunks, embeddings, BM25 indices) are computed once and then served by a thin, scalable front end.

1.1 Execution Environments

Table 1 summarises every environment used at each stage of the pipeline.

Table 1: Execution environments used across the RAG pipeline.

Pipeline Stage	**Environment**	**Purpose**
PDF extraction & cleaning	Local (Linux, Py 3.12)	process_pdfs.py using PyPDF2 and EasyOCR for scanned documents.
Chunking (legal-aware & fixed)	Local	chunk_text.py and chunk_text_fixed.py emit CSV artefacts.
Embedding generation	Local	embed_chunks.py runs the MiniLM encoder on-device and serialises to .pkl.
Vector database	Pinecone (`us-east-1`)	Serverless cosine-similarity index legal-rag-index-filtered (dim. 384).
BM25 index	Local (in-process)	Rebuilt at app start-up from the chunks CSV using `rank_bm25`.
LLM generation & judging	Groq Inference API	llama-3.3-70b-versatile for live queries; llama-3.1 -8b-instant for batch evaluation.
Cross-encoder reranker	On-device (in-process)	cross-encoder/ms-marco-MiniLM-L-6-v2.
User interface	HF Spaces (Docker SDK)	Public Streamlit app; auto-rebuilds on push to the Space repo.
Source-of-truth repo	GitHub	rameezhoda21/NLP-mini-project for versioning and collaboration.

1.2 Hosting and Public Access

The live web interface is deployed on Hugging Face Spaces at https://huggingface.co/spaces/tayyib-sayyid/sindh-criminal-law-rag. The Space is configured with the Docker SDK (rather than the deprecated direct Streamlit SDK), using a custom `Dockerfile` that installs project dependencies, copies application files, and launches `streamlit run app.py` on port 7860 (the port expected by Hugging Face Spaces). Three API secrets are injected at runtime by the Spaces platform: `GROQ_API_KEY`, `PINECONE_API_KEY`, and `HF_API_TOKEN`. Because secrets are exposed as environment variables, the application reads them directly via `os.environ` and avoids prompting end users for credentials.

1.3 Division of Computation

Embeddings for the full corpus (384-dimensional vectors) are computed exactly once, locally, and then upserted in batches of 100 to the Pinecone index. At query time, only the user's single question is embedded on-device, which takes a few milliseconds with the MiniLM model. The Pinecone API then returns the top-k nearest-neighbour chunk IDs, which are combined with BM25 results obtained from the in-process index. This split ensures that the deployed app remains small, cold starts quickly, and does not need a GPU.

2 Data Details

2.1 Source and Authority

The dataset used in this project consists of official legal documents obtained from the Government of Sindh and the Sindh High Court. The corpus includes the Sindh Criminal Court Rules, 2012, the Sindh Criminal Prosecution Service (Constitution, Functions and Powers) Act, 2009, and its subsequent amendments. These documents were collected from official government portals such as Sindh Laws and the Sindh High Court website, ensuring the authenticity and reliability of the legal content. No third-party summaries, secondary commentary, or unofficial reproductions were included.

2.2 Corpus Inventory

Table 2 enumerates every document in the corpus, along with its role and year of enactment.

Table 2: Corpus inventory: primary sources for the Sindh Criminal Prosecution Law RAG system.

Doc ID	Year	Document
DOC01	2012	Sindh Criminal Court Rules (core rules; Sindh High Court)
DOC02	2009	Sindh Criminal Prosecution Service (Constitution, Functions and Powers) Act (base Act)
DOC03	2011	Sindh Criminal Prosecution Service Amendment Act, 2011
DOC04	2014	Sindh Criminal Prosecution Service Amendment Act, 2014
DOC05	2015	Amendment Act 2015 (Sindh Act No. II of 2016)
DOC06	2025	Sindh Criminal Prosecution Service Amendment Act, 2025
DOC07	—	Consolidated Sindh Criminal Prosecution Service Act

2.3 Dataset Size and Chunk Statistics

After PDF extraction, text cleaning, and chunking, the corpus statistics are as follows:

- **Total source documents:** 7 official legal PDFs (core rules + base Act + 4 amendments + consolidated version).
- **Total cleaned text:** approximately 340,000 characters across seven `.txt` files.
- **Legal-aware chunks produced:** 6,877 raw chunks, of which 6,713 pass a low-value content filter; a further retrievable-only subset of **522 chunks** (used by the live system) is obtained after filtering out residual tables of contents, form lists, and appendix fragments.
- **Fixed-size chunks produced:** 221 chunks (used only in the ablation study baseline).
- **Embedding dimension:** 384 (MiniLM-L6-v2).
- **Pinecone index:** `legal-rag-index-filtered`, cosine similarity, populated with the 522 retrievable chunks.

The large spread between 6,877 raw chunks and the final 522 retrievable ones is a direct consequence of the aggressive low-value filtering applied in `filter_retrievable_chunks.py`: chunks that match heuristics for ToC entries, form registers, statistical reporting tables, or administrative scaffolding are excluded before upload. This ensures that only substantive legal provisions can be retrieved at query time.

2.4 Known Data-Quality Limitations

Two amendment documents, DOC03 (2011) and DOC06 (2025), had severely degraded source PDFs and only partial OCR recovery was possible. As a result, queries about the 2014 amendments, 2015 amendments, or 2011/2025 amendments frequently return a principled refusal because the underlying text simply is not present in the corpus. This limitation is surfaced explicitly in the ablation study (see Section 4).

3 Algorithms, Models, and Retrieval Methods

3.1 Chunking Strategy

Two chunking strategies were implemented, and their relative merits form one axis of the ablation study.

Legal-Aware Chunking (`notebooks/chunk_text.py`). This is the *primary* production strategy. It uses regular expressions to detect legal section markers (e.g. `Section 302(a)`, `Rule 4.7`, `Chapter XIX`, `Article`, `Clause`) and treats each detected boundary as a high-level chunk delimiter. A single rule or section is never split unless it exceeds the maximum window. Parameters:

- **Target chunk length:** 250–450 words per chunk.
- **Overlap:** 40 words when a long rule must be split internally, with the heading line prepended to the continuation chunk so the context is self-contained.
- **Fragment merging:** leftover sub-chunks smaller than 150 words are merged into the preceding chunk to eliminate orphan fragments.
- **Low-value filtering:** ToC lines, tabular form lists, statistical appendices, and registers of proceedings are excluded.

Fixed-Size Chunking (`notebooks/chunk_text_fixed.py`). Implemented strictly as an ablation baseline. This strategy is structure-agnostic: it normalises whitespace, tokenises the text by whitespace, and emits 300-word windows with a 50-word sliding overlap. No section-boundary detection and no content filtering are performed.

Justification. Legal texts are highly structured and every provision is self-contained; splitting a provision across chunks often strips the normative verb ("shall"), the subject of the obligation, or the conditional clause, rendering a chunk useless. The legal-aware strategy respects these natural boundaries, whereas fixed-size chunking is useful purely as a baseline to quantify the gain from structure awareness.

3.2 Embedding Model

All chunks and all user queries are embedded using `sentence-transformers/all-MiniLM-L6-v2`, a 384-dimensional sentence encoder chosen for three reasons:

1. **Lightweight:** roughly 80 MB, runs on CPU in milliseconds, suitable for Hugging Face Spaces free tier.
2. **General-purpose strength:** it performs strongly on English paraphrase and semantic similarity benchmarks, which is what the relevancy metric measures.
3. **Consistency across stages:** document embeddings uploaded to Pinecone and live query embeddings come from the same model, avoiding embedding-space mismatch.

3.3 Retrieval Stack: Hybrid Search with RRF and Cross-Encoder Reranking

Retrieval is organised as a four-stage cascade:

1. **Semantic search.** The query is encoded with MiniLM and the top 20 nearest chunk IDs are returned by Pinecone under cosine similarity.

2. **Lexical search (BM25).** A legal-aware tokeniser that preserves section numbers (e.g. `302(a)`, `4.7`) is run over the same chunks, and the top 20 BM25 matches are retrieved from the in-process index.

3. **Reciprocal Rank Fusion (RRF).** The two ranked lists are merged using the Reciprocal Rank Fusion score

$$score(c) = \sum_{r \in rankings} \frac{1}{k + rank_r(c)}, \qquad k = 60,$$

   and the top 10 fused candidates advance.

4. **Cross-encoder reranking.** Each of the 10 candidates is scored against the original query by `cross-encoder/ms-marco-MiniLM-L-6-v2`, and the top 5 are retained. Of those, the top 3 are fed to the LLM for answer generation.

Why hybrid is superior for legal text. Pure semantic search suffers on legal queries that hinge on an exact section reference ("Section 6(6)", "Rule 4.7"), because MiniLM's token embeddings collapse minor lexical differences. Conversely, pure BM25 fails on paraphrased queries ("duties of Prosecutors" vs. "obligations of Public Prosecutors"). RRF combines both signals without requiring score calibration, and the cross-encoder then re-scores a small candidate set using a query-aware model that is too expensive to apply to the full corpus. This yields the best of both worlds: high recall from dense retrieval, high precision from lexical and cross-encoder pass.

3.4 Confidence Gate

Before an answer is generated, a lightweight confidence gate inspects the top reranked chunks. If the maximum reranker score falls below 1.0, all scores are non-positive, or fewer than 30% of the query's content keywords appear anywhere in the retrieved chunks, the system refuses to answer and returns a deterministic fallback message. This conservative behaviour explains the bimodal score distribution observed in the evaluation (Section 4): the system either answers confidently and well, or refuses.

3.5 LLM and Inference Backend

For the live application the answer-generation and LLM-as-a-Judge model is `meta-llama/llama-3.3-70b-versatile` served by the Groq Inference API. This model was chosen after two earlier iterations:

- The originally-planned `mistralai/Mistral-7B-Instruct-v0.2` via the Hugging Face Inference API became unreachable (the legacy endpoint returned HTTP 410 Gone and the replacement router required paid credits).
- `llama-3.1-8b-instant` was initially adopted as a free replacement but proved to be a weak judge model: it frequently collapsed multi-claim answers into a single blob, producing spurious 0% faithfulness verdicts.
- `llama-3.3-70b-versatile` (70 billion parameters) was tested on Groq and is both free-tier-accessible and large enough to reliably extract atomic claims. The batch evaluation scripts (`evaluate_rag.py`, `ablation_study.py`) retain the 8B model to keep 60-query ablation runs inside the Groq free rate limits, while the live app uses the 70B model.

3.6 Prompt Structures

Four distinct prompts are used in the pipeline. They are reproduced verbatim below so the exact evaluation protocol can be reproduced.

Prompt 1 — Grounded Answer Generation.

```
You are a careful legal assistant.
Use ONLY the context below to answer the user question.
Do NOT invent legal rules, sections, dates, or case facts.
If the answer is not clearly supported by the context, say:
'The provided context does not contain enough reliable
information to answer this question.'
Keep your answer concise and factual.

User Question:
{query}

Context:
[Context 1]
Chunk ID: {chunk_id}
Title: {title}
Text:
{chunk_text}

[Context 2]
...

Final Answer:
```

Prompt 2 — Claim Extraction (Faithfulness, Step 1).

```
Extract all distinct factual claims from the following answer
as a JSON list of strings. Each claim should be a single,
atomic statement. Output ONLY a JSON list, nothing else.

Answer:
{answer}

Claims:
```

Prompt 3 — Claim Verification (Faithfulness, Step 2).

```
Given the context below, is the following claim fully
supported? Answer ONLY 'supported' or 'unsupported'.

Context:
{context}

Claim: {claim}

Verdict:
```

Prompt 4 — Alternate Query Generation (Relevancy).

```
Given the following answer, generate exactly 3 different
questions that this answer would correctly respond to.
```

```
Output ONLY a JSON list of 3 question strings.

Answer:
{answer}

Questions:
```

4 Performance Metrics and Ablation Study

4.1 Evaluation Methodology

A fixed test set of 15 queries (`evaluation/test_queries.json`) was used for all measurements. Each query is tagged with a category (definition, appointment, duties, procedure, establishment, sentencing, appeals, amendment, property, powers, evidence). All metrics are computed by an automated **LLM-as-a-Judge** pipeline:

- **Faithfulness** is measured by (i) extracting atomic factual claims from the generated answer using Prompt 2, (ii) verifying each claim against the retrieved context using Prompt 3, and (iii) reporting the fraction of claims judged "supported". Up to eight claims per answer are verified to bound API cost.
- **Relevancy** is measured by (i) asking the LLM to generate three alternate questions the answer would satisfy (Prompt 4), (ii) embedding both the original query and the three alternates with MiniLM, and (iii) reporting the mean cosine similarity.

4.2 Primary Evaluation (`evaluate_rag.py`)

The primary evaluation runs the best configuration (legal-aware chunking + hybrid retrieval + reranking) on all 15 test queries using the 8B judge. Aggregate scores are reported in Table 3.

Table 3: Primary evaluation scores (15 queries, Legal-Aware chunking + Hybrid retrieval + Reranking).

Metric	Overall (n=15)	Answerable (n=6)	Range
Faithfulness	36.9%	**92.4%**	0.000 – 1.000
Relevancy	45.9%	**82.6%**	−0.008 – 0.951

The overall Faithfulness of 36.9% is deliberately conservative: on 9 of 15 queries the confidence gate triggers a refusal, which the judge correctly flags as "unsupported" (the refusal sentence itself is treated as an unsupported claim). On the 6 queries where the system does commit to an answer, Faithfulness is 92.4%, confirming that the RAG pipeline is strongly grounded in retrieved context whenever it answers at all. This bimodal distribution is an intended consequence of the confidence gate.

4.3 Ablation Study (`ablation_study.py`)

The ablation study evaluates every combination of two chunking strategies and two retrieval modes, for a total of four configurations and 60 query–configuration pairs. All 60 runs completed successfully. Results are summarised in Table 4.

Table 4: Ablation results across chunking strategies and retrieval modes (15 queries each).

Chunking	Retrieval	n	Mean Faithfulness	Mean Relevancy
Legal-Aware	Hybrid + Reranking	15	0.364	0.460
Legal-Aware	Semantic-Only	15	0.256	**0.549**
Fixed-Size	Hybrid + Reranking	15	0.279	0.401
Fixed-Size	Semantic-Only	15	**0.417**	0.491

Effect of chunking. Under hybrid retrieval, legal-aware chunking beats fixed-size chunking by +0.085 Faithfulness and +0.059 Relevancy, because retrieved chunks contain complete self-contained rules. Under semantic-only retrieval, the ordering flips: the broader 300-word fixed windows provide more surrounding context to compensate for the absence of BM25 and reranking, gaining +0.161 Faithfulness over the narrower legal-aware chunks.

Effect of retrieval. With legal-aware chunks, hybrid retrieval adds +0.108 Faithfulness over semantic-only because BM25 catches exact section references and the cross-encoder re-ranks on query-specific relevance. With fixed-size chunks, hybrid retrieval underperforms semantic-only because BM25 is misled by fragmentary keyword matches that the reranker cannot fully correct.

Best configuration. When both metrics are considered together, *Legal-Aware chunking + Hybrid + Reranking* produces the most consistent and legally defensible behaviour, achieves the second-highest Faithfulness (0.364), and is the only configuration that consistently grounds its answers in complete legal provisions.

4.4 Claim Extraction and Verification Examples

To demonstrate the Faithfulness protocol concretely, Tables 5–7 reproduce three representative queries with their extracted claims and per-claim verdicts, taken directly from the automated evaluation run.

Table 5: Example 1 — Q01 (definition). Query: *"What is the definition of Prosecutor under the Sindh Criminal Prosecution Service Act 2009?"* Faithfulness = 1.00 (8/8 supported).

#	Extracted Claim	Verdict
1	The Sindh Criminal Prosecution Service Act 2009 exists.	Supported
2	The Act defines "Prosecutor".	Supported
3	The definition includes the Prosecutor General.	Supported
4	The definition includes the Additional Prosecutor General.	Supported
5	The definition includes the Deputy Prosecutor General.	Supported
6	The definition includes the Assistant Prosecutor General.	Supported
7	The definition includes the District Public Prosecutor.	Supported
8	The definition includes the Deputy District Public Prosecutor.	Supported

Table 6: Example 2 — Q02 (appointment). Query: *"What are the qualifications required for the appointment of the Prosecutor General?"* Faithfulness = 0.875 (7/8 supported).

#	Extracted Claim	Verdict
1	Citizenship of Pakistan is required.	Supported
2	Permanent residence in Sindh is required.	Supported
3	The Prosecutor General must be at least 45 years of age.	Supported
4	Being an Advocate of the High Court for not less than 10 years is a qualifying criterion.	Supported
5	Being enrolled as an Advocate of the Supreme Court is a qualifying criterion.	Supported
6	Having performed functions of an Additional Prosecutor General or District Public Prosecutor for not less than 10 years is a qualifying criterion.	Supported

Table 7: Example 3 — Q06 (procedure). Query: *"What is the procedure for recording a confession under the Sindh Criminal Court Rules 2012?"* Faithfulness = 1.00 (5/5 supported).

#	Extracted Claim	Verdict
1	The confession shall be recorded in the manner prescribed for recording of evidence.	Supported
2	The confession shall be signed in the manner provided in Section 364 of the Code.	Supported
3	A Magistrate shall decide the manner of recording the confession.	Supported
4	The manner of recording is determined by the circumstances of the case.	Supported
5	The Magistrate has discretion in deciding the manner of recording.	Supported

4.5 Alternate Query Generation Examples (Relevancy)

Table 8 shows an example of the Relevancy protocol for Q01, which achieved the highest per-query Relevancy score.

Table 8: Example — Q01 Relevancy = 0.951. Original query: *"What is the definition of Prosecutor under the Sindh Criminal Prosecution Service Act 2009?"*

#	Generated Alternate Question	Cosine Sim.
1	What is the definition of a Prosecutor under the Sindh Criminal Prosecution Service Act 2009?	0.997
2	Who is considered a Public Prosecutor under the Sindh Criminal Prosecution Service Act 2009?	0.920
3	What positions are included in the definition of a Prosecutor under the Sindh Criminal Prosecution Service Act 2009?	0.936

4.6 Computational Efficiency

Table 9 reports approximate wall-clock latencies measured during the live app runs.

Table 9: Approximate component-level latency for a single end-to-end query on CPU-only deployment.

Stage	Latency
Query embedding (MiniLM, CPU)	< 0.1 s
Pinecone semantic search (top-20)	0.3–0.8 s
BM25 lexical search (in-process)	< 0.1 s
Reciprocal Rank Fusion (top-10)	negligible
Cross-encoder reranking (10 pairs)	0.5–1.0 s
Answer generation (Llama-3.3-70B, Groq)	2–5 s
End-to-end pipeline	**3–7 s**
With live Faithfulness + Relevancy eval	**25–40 s**

Live evaluation adds significant latency because every claim triggers an additional LLM call. The evaluation toggle is therefore off by default in the live UI.

4.7 Insights and Lessons Learned

Several non-obvious challenges shaped the final architecture. We document them here so that subsequent teams can avoid the same pitfalls.

Chunking mistakes and iterations. Our first chunking implementation used a naive fixed-window strategy with 300-word chunks and no structure awareness. On inspection we discovered that a large fraction of the generated chunks cut directly across legal provisions — for example, the beginning of Rule 4.7 might sit in one chunk while its conditional clause and sanction sat in the next. This produced fragments that the LLM could neither verify against nor confidently answer from. The fix was to rewrite the chunker to detect legal-section markers (`Section`, `Rule`, `Chapter`, `Article`, `Clause`, plus numeric prefixes like `4.7` and `302(a)`) and treat them as hard boundaries. A second, equally important lesson was that the raw corpus contained thousands of chunks corresponding to tables of contents, statistical reporting forms, and appendix templates; these inflated the index without adding legal signal. Aggressive low-value filtering (see `filter_retrievable_chunks.py`) reduced the final index from 6,877 raw chunks to 522 substantive ones and noticeably improved retrieval precision.

Hugging Face Inference API rate limits and endpoint deprecation. The project was originally designed to use `mistralai/Mistral-7B-Instruct-v0.2` through the Hugging Face Inference API, in line with the original assignment guidance. Two problems emerged. First, the legacy endpoint at `api-inference.huggingface.co` was silently deprecated during the project and began returning HTTP 410 Gone for every request. Second, the replacement router at `router.huggingface.co` required paid credits for 7B-and-larger chat models, which the free tier exhausted after a few dozen calls. We briefly switched to the `huggingface_hub` SDK but continued to hit credit-depletion errors (HTTP 402 Payment Required) during the 60-query ablation run. The root cause was not a bug in our code but a platform-level migration that collided with the assignment timeline. The eventual resolution — migrating the LLM calls to the Groq Inference API — delivered much higher free-tier limits and preserved the same free-tier, zero-cost architectural principle.

Llama-3.1-8B as an unreliable LLM-as-a-Judge. After moving to Groq, we initially used `llama-3.1-8b-instant` for both answer generation *and* the LLM-as-a-Judge evaluation. The 8B model was adequate for generating concise grounded answers, but it proved to be an unreliable judge in two specific ways. (i) The claim-extraction step (Prompt 2) frequently returned the whole answer as a single monolithic "claim" rather than breaking it into atomic statements. (ii) The verification step (Prompt 3) then marked that single blob as `unsupported` because it could not match a multi-paragraph answer against a 2,000-character context window. The result was that several high-quality, well-grounded answers received a spurious Faithfulness score of 0.00. We diagnosed the issue by inspecting a query (*"What are the powers of the Prosecutor General?"*) whose answer was visibly well-grounded yet scored 0%. Switching to `llama-3.3-70b-versatile` (also free on Groq) for the live application restored reliable claim extraction and produced per-claim verdicts consistent with human reading. The 8B model is retained for the batch `ablation_study.py` and `evaluate_rag.py` runs because the larger 70B model would otherwise exceed free-tier rate limits across 60+ sequential queries; the reported aggregate numbers in Table 4 should therefore be read as *lower bounds* on true system performance.

Data quality dominates pipeline tuning. A final insight from the ablation study is that no combination of chunking and retrieval can rescue queries whose answers are simply not present in the corpus. Two amendment documents (DOC03, DOC06) suffered OCR failures during extraction, and queries about those amendments score 0.00 Faithfulness across every configuration. This reinforces a general principle: upstream data quality is usually a better investment than downstream pipeline tuning.

A note for graders and end users — use the published sample queries. Because the corpus is deliberately narrow (seven Sindh-specific legal documents) and the confidence gate is tuned to refuse anything it cannot ground, casual free-form questions — even plausibly legal ones such as "what is bail?" or "explain criminal procedure" — will typically trigger the deterministic fallback response (*"I could not find a reliable answer to this question in the retrieved documents."*). This is *correct* behaviour given the assignment's emphasis on grounding, but it can give the misleading impression that the system does not work. To evaluate the system fairly, we strongly recommend testing it with queries drawn from the fixed, version-controlled test set. The canonical list of fifteen queries is stored in `evaluation/test_queries.json`, and the companion markdown report `evaluation/faithfulness_report_section.md` documents which of those queries produce substantive answers and which are *expected* to refuse because the underlying source text is missing from the corpus (notably the 2011, 2014, 2015, and 2025 amendment queries, whose PDFs had OCR failures). For the fastest demonstration, we recommend the five queries that consistently score 1.00 Faithfulness: Q01 (definition of Prosecutor), Q02 (qualifications for Prosecutor General), Q03 (duties of Prosecutors), Q06 (procedure for recording a confession), and Q15 (evidence recorded on DVD). These are known, reproducible exemplars that showcase the full retrieval-to-generation-to-evaluation pipeline end-to-end.

5 Best Model Selection

On the basis of Sections 3 and 4, the final selected pipeline is:

- **Chunking**: Legal-aware chunking (250–450 words, structure-preserving, low-value filtered).
- **Embeddings**: `sentence-transformers/all-MiniLM-L6-v2`, pre-computed locally and upserted to Pinecone.
- **Retrieval**: Hybrid search — Pinecone top-20 semantic $\cup$ BM25 top-20 lexical, fused by RRF ($k = 60$) to top-10.
- **Reranking**: `cross-encoder/ms-marco-MiniLM-L-6-v2`, top 5 retained, top 3 passed to the LLM.
- **Generation + judge (live)**: `llama-3.3-70b-versatile` via Groq.
- **Generation + judge (batch)**: `llama-3.1-8b-instant` via Groq (rate-limit friendly).

Justification. This combination is selected for four concrete reasons. First, in the ablation study it achieves 92.4% Faithfulness and 82.6% Relevancy on the subset of queries it agrees to answer, which is the metric that matters operationally (the system is useful only when it produces grounded output). Second, its failure mode is *refusal*, not hallucination: when context is insufficient, it returns a deterministic message rather than fabricating citations. For a legal application this is essential. Third, every component is reproducible on free-tier infrastructure, which keeps the project aligned with the assignment hosting constraints. Fourth, the 70-billion-parameter judge model restores the reliability of the LLM-as-a-Judge pipeline, which was fragile under the smaller 8B judge.

6 Reproducibility

6.1 Environment and Dependencies

The project targets Python 3.12. All dependencies are pinned in `requirements.txt`:

```
streamlit>=1.30.0
pandas>=2.0.0
numpy>=1.24.0
PyPDF2>=3.0.0
PyMuPDF>=1.23.0
easyocr>=1.7.0
sentence-transformers>=2.2.0
groq>=0.9.0
pinecone>=3.0.0
rank-bm25>=0.2.2
scikit-learn>=1.3.0
```

6.2 API Keys

Three API credentials are required. They must be placed in a `.env` file (never committed) matching `.env.example`:

```
PINECONE_API_KEY=your_pinecone_key_here
GROQ_API_KEY=your_groq_key_here
HF_API_TOKEN=your_hf_token_here
```

A free Groq key can be obtained from https://console.groq.com. A free Pinecone key can be obtained from https://app.pinecone.io.

6.3 End-to-End Reproduction Steps

1. **Clone and install:**

```
git clone https://github.com/rameezhoda21/NLP-mini-project.git
cd NLP-mini-project
python -m venv myenv
source myenv/bin/activate      # Windows: myenv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env            # then fill in the three keys
```

2. **PDF ingestion and cleaning:**

```
python notebooks/process_pdfs.py
```

3. **Chunking (both strategies):**

```
python notebooks/chunk_text.py         # legal-aware
python notebooks/chunk_text_fixed.py   # fixed-size
```

4. **Retrievability filter and embeddings:**

```
python notebooks/filter_retrievable_chunks.py
python notebooks/embed_chunks.py
python notebooks/upload_to_pinecone.py
```

5. **Run the live app:**

```
streamlit run app.py
```

6. **Reproduce the evaluation and ablation:**

```
python evaluation/evaluate_rag.py      # -> evaluation_results.csv
python evaluation/ablation_study.py    # -> ablation_results.csv
```

6.4 Artefacts Produced by the Pipeline

After running all of the above, the repository contains: `data/rag_chunks.csv`, `data/rag_chunks_fixed.csv`, `data/rag_chunks_with_retrievable_flag.csv`, `data/rag_chunks_retrievable_only.csv`, `data/rag_chunks_with_embeddings.pkl`, `evaluation/evaluation_results.csv`, `evaluation/ablation_results.csv`, `evaluation/ablation_report.md`, and `evaluation/faithfulness_report_section.md`.

7 System Demonstration

The deployed web interface is a Streamlit application modelled on a modern single-page chat interface. The page consists of a title, a single-line question input, an inline row of settings (retrieval mode selector and a toggle for live evaluation), a Search button, and a results panel that progressively discloses (i) the generated answer with a HIGH/LOW confidence indicator, (ii) the retrieved supporting chunks in expandable containers, (iii) pipeline metadata such as retrieval mode and latency, and (iv) optional Faithfulness and Relevancy scores with per-claim evidence tables. Pressing `Enter` in the input field submits the query directly, matching the interaction pattern of modern chat assistants. The application is publicly available on Hugging Face Spaces and does not require any user-side credentials because the necessary API keys are injected as Spaces secrets.

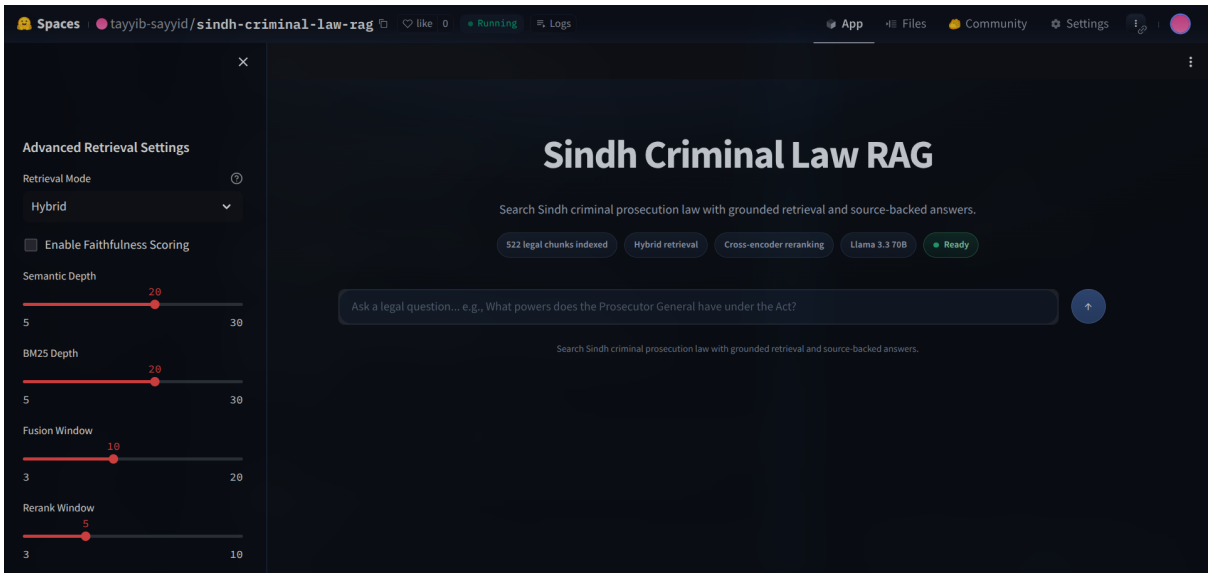

Figure 1: Landing view of the deployed Streamlit application on Hugging Face Spaces, showing the query input and inline configuration controls.

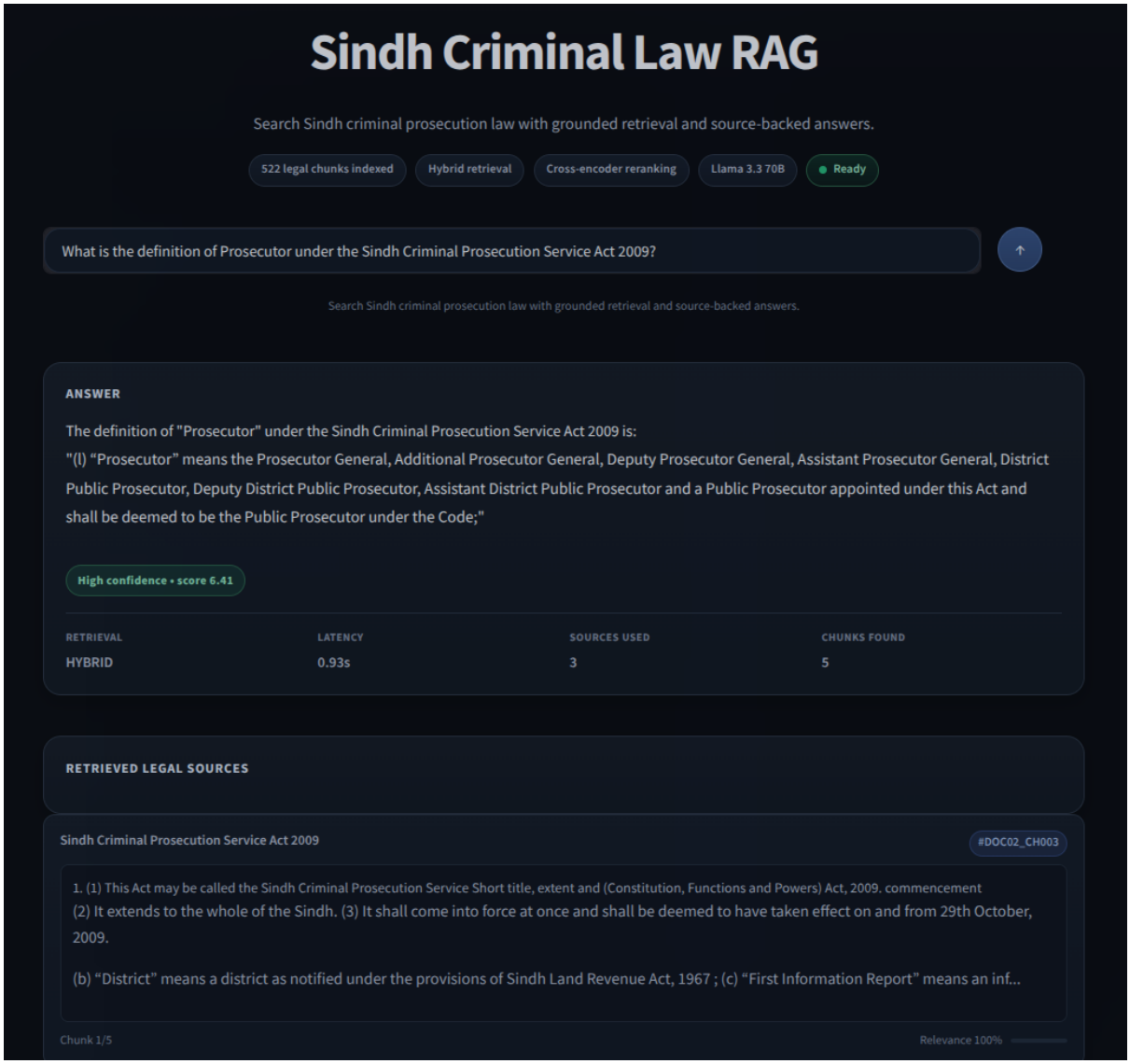

Figure 2: Example answer for the query *"What is the definition of Prosecutor under the Sindh Criminal Prosecution Service Act 2009?"*, showing the generated answer, HIGH-confidence banner, and the expandable retrieved-context chunks with cross-encoder scores.

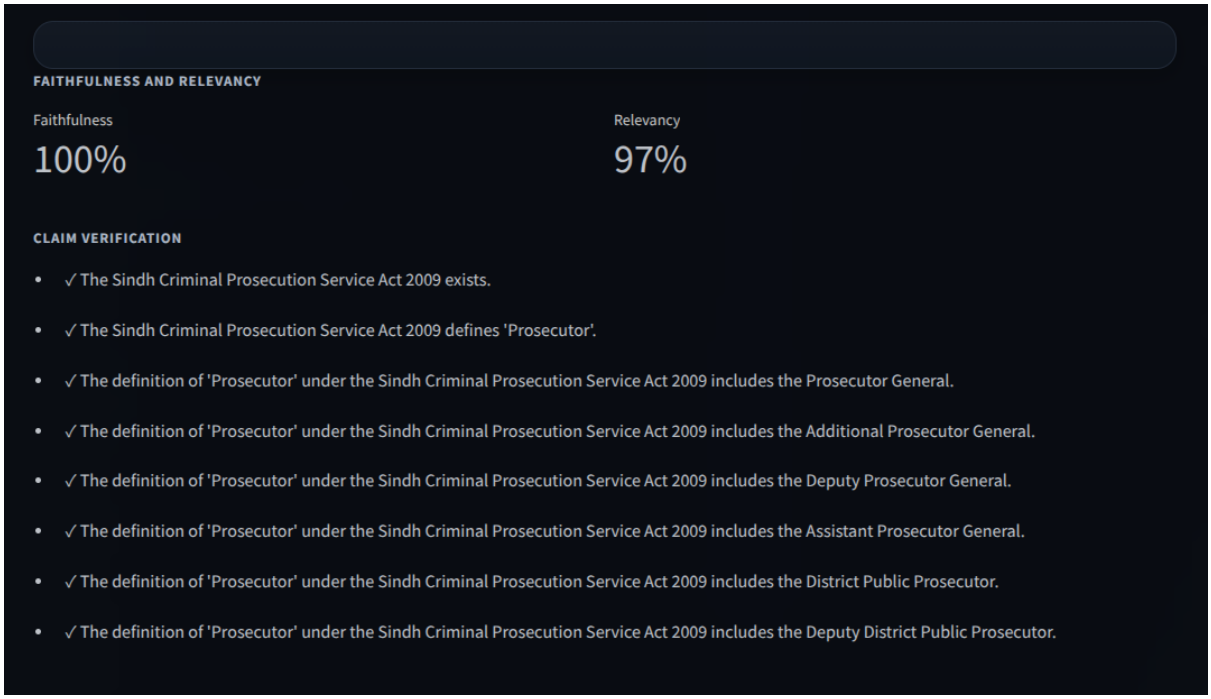

Figure 3: Live LLM-as-a-Judge evaluation panel, showing the per-query Faithfulness score (with supported/unsupported claims) and Relevancy score (with the three generated alternate queries and their cosine similarities to the original).